means that user-provided keys are treated as passwords, and it requires you to implement server routes that act as intermediaries between user requests and external APIs.

While allowing client storage of API keys can reduce server load and simplify implementation, it poses significant risks that cannot be overlooked. One of the primary concerns is the exposure of sensitive information through browser vulnerabilities. When API keys are embedded in client code, they become accessible to anyone who inspects the web application using developer tools. Some providers, like OpenAI, have built-in detection and may automatically disable any API key that has been exposed in that way.

Moreover, relying on client API calls lacks usage control and monitoring capabilities. Without a server intermediary, it becomes challenging to track how the API keys are being used, making it difficult to identify and mitigate abuse. This lack of oversight can result in unexpected charges if the keys are exploited for excessive usage, which will anger your users.

Additionally, implementing this approach is not trivial. It requires the client to import all necessary SDKs to perform API calls directly from the client. This not only complicates the setup but also necessitates exposing some internal logging, like internal prompt queries and processing pipelines, to make it work effectively.

For all the reasons mentioned, it is preferable to manage your own organization's API keys when exposing a service to the public. This approach ensures that you have ultimate responsibility and control over API usage, which is critical for maintaining security and compliance.

Many providers, like OpenAI or Google AI, offer robust API usage-monitoring capabilities through their platforms. For example, Google Cloud's Vertex AI allows users to monitor API usage effectively, particularly for the Gemini API. OpenAI has a Usage tab, showing how its API and models are used from within the organization (figure 9.5).

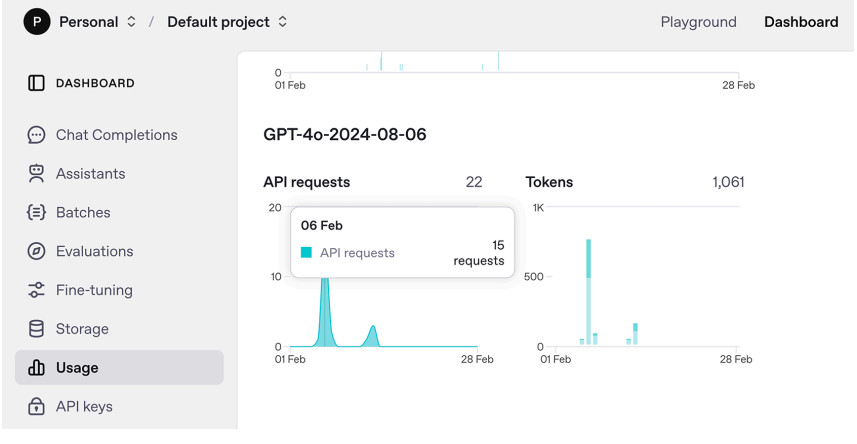


Figure 9.5 OpenAI API usage dashboard displaying metrics for a GPT-4 model instance, showing API requests and total token usage. The dashboard interface includes a navigation sidebar with options for monitoring various aspects of the API deployment.